

Testing de Software

Valeria Bengolea

Departamento de Computación

Facultad de Ciencias Exactas, Físico-Químicas y Naturales

Universidad Nacional de Río Cuarto

Testing basado en propiedades

Property Based Testing

- Un test unitario captura el comportamiento del SUT para una entrada particular;
- Los test basados en propiedades captura algún aspecto o propiedad del comportamiento del SUT para un número **potencialmente infinito de entradas**

Property Based Testing

{**pre**: !q.isFull() }

q.enqueue()

{**pos**: q.size > 0 }

Property based Testing

Una propiedad:

```
public void myProperty(MyEntry e) {  
    AssumePrecondition(e);  
    Arrange(e);  
    Act(e);  
    AssertExpectedProperty(e);  
}
```

Property based Testing

Una propiedad:

```
public void myProperty(MyEntry e) {  
    AssumePrecondition(e);  
    Arrange(e);  
    Act(e);  
    AssertExpectedProperty(e);  
}
```

Precondición: Descripción de las entradas para las cuales la propiedad es válida

Property based Testing

Una propiedad:

```
public void myProperty(MyEntry e) {  
    AssumePrecondition(e);  
    Arrange(e);  
    Act(e);  
    AssertExpectedProperty(e);  
}
```

Precondición: Descripción de las entradas para las cuales la propiedad es válida

Postcondición:
Propiedad que se espera que el código satisfaga, para valores cualesquiera de los parámetros que cumplan con la precondición

Property Based Testing

Ejemplos de propiedades:

“Para toda cola no llena, insertar un elemento cualquiera, incrementa en 1 su tamaño”

“ Para todo set, si se agrega un elemento que ya pertenece al set entonces el set no cambia”

“Para todo a y b strings, a es substring de concat(a,b)”.

Para todo par de enteros a y b, `max(int, int): int` cumple con:

- `Math.max(a, b) = Math.max(b, a)`
- `Math.max(a, b) >= a`
- `Math.max(a, b) >= b`
- `p + Math.max(a,b) = Math.max(p+a, p+b)`

Testing basado en propiedades

Usualmente un test basado en propiedades está compuesto por:

- **Precondición:** Descripción de las entradas para las cuales la propiedad es válida
 - Parámetros que violan la precondición son ignorados
- **Pieza de código a testear** (método, módulo, etc..)
- **Postcondición:** Propiedad que se espera que el código satisfaga, para valores cualesquiera de los parámetros que cumplan con la precondición

Property Based Testing

“Para toda cola no llena, insertar un elemento cualquiera, incrementa en 1 su tamaño”

Property Based Testing

“Para toda cola no llena, insertar un elemento cualquiera, incrementa en 1 su tamaño”

```
@Test
public void enqueueSizeTest_1() {
    BoundedQueue q = new BoundedQueue();
    q.enqueue(4);
    q.enqueue(3);
    q.enqueue(5);
    assertThat(q.size(), is(3));
}
```

Property Based Testing

“Para toda cola no llena, insertar un elemento cualquiera, incrementa en 1 su tamaño”

```
@Test
public void enqueueSizeTest_1() {
    BoundedQueue q = new BoundedQueue(4);
    q.enqueue(4);
    q.enqueue(3);
    q.enqueue(5);
    assertThat(q.size(), is(3));
}
```

```
@Test
public void enqueueSizeTest_1() {
    BoundedQueue q = new BoundedQueue();
    q.enqueue(4);
    q.enqueue(3);
    assertThat(q.size(), is(2));
}
```

Property Based Testing

“Para toda cola no llena, insertar un elemento cualquiera, incrementa en 1 su tamaño”

```
@Test
public void enqueueSizeTest_1() {
    BoundedQueue q = new BoundedQueue(4);
    q.enqueue(4);
    q.enqueue(3);
    q.enqueue(5);
    assertThat(q.size(), is(7));
}
```

```
@Test
public void enqueueSizeTest_1() {
    BoundedQueue q = new BoundedQueue();
    q.enqueue(4);
    q.enqueue(3);
    assertThat(q.size(), is(7));
}
```

```
@Test
public void enqueueSizeTest() {
    BoundedQueue q = new BoundedQueue();
    q.enqueue(4);

    int oldSize = q.size();
    q.enqueue(3);

    assertThat(q.size(), is(oldSize + 1));
}
```

Property Based Testing

“Para toda cola no llena, insertar un elemento cualquiera, incrementa en 1 su tamaño”

```
@Test
public void enqueueSizeTest_1() {
    BoundedQueue q = new BoundedQueue(4);
    q.enqueue(4);

    q.enqueue(3);
    q.enqueue(5);

    assertThat(q.size(), is(6));
}
```

```
@Test
public void enqueueSizeTest_1() {
    BoundedQueue q = new BoundedQueue();
    q.enqueue(4);
    q.enqueue(3);

    assertThat(q.size(), is(7));
}
```

```
@Test
public void enqueueSizeTest() {
    BoundedQueue q = new BoundedQueue();
    q.enqueue(4);

    int oldSize = q.size();
    q.enqueue(3);

    assertThat(q.size(), is(oldSize + 1));
}
```

Sobre instancias de BoundedQueue particulares

Property Based Testing

Ejemplo

“Para toda cola no llena, insertar un elemento cualquiera, incrementa en 1 su tamaño”

```
public void enqueueSizeProperty(BoundedQueue queue, Integer i) {  
    assumeTrue("cola no llena", !queue.isFull());  
  
    int oldSize = queue.size();  
    queue.enqueue(i);  
  
    assertThat(queue.size(), is(oldSize + 1));  
}
```

Property Based Testing

Ejemplo

“Para toda cola no llena, insertar un elemento cualquiera, incrementa en 1 su tamaño”

Precondición: Descripción de las entradas para las cuales la propiedad es válida

```
public void enqueueSizeProperty(BoundedQueue queue, Integer i) {  
    assumeTrue("cola no llena", !queue.isFull());  
    int oldSize = queue.size();  
    queue.enqueue(i);  
    assertThat(queue.size(), is(oldSize + 1));  
}
```

Property Based Testing

Ejemplo

“Para toda cola no llena, insertar un elemento cualquiera, incrementa en 1 su tamaño”

Precondición: Descripción de las entradas para las cuales la propiedad es válida

```
public void enqueueSizeProperty(BoundedQueue queue, Integer i) {  
    assumeTrue("cola no llena", !queue.isFull());  
  
    int oldSize = queue.size();  
    queue.enqueue(i);  
  
    assertThat(queue.size(), is(oldSize + 1));  
}
```

Postcondición:
Propiedad que se espera que el código satisfaga, para valores cualesquiera de los parámetros que cumplan con la precondición

Property Based Testing

Ejemplo

“ Para todo set, si se agrega un elemento que ya pertenece al set entonces el set no cambia”

```
void addAlreadyBelongsToDoesNotExtendsSet(BitSet set, int i)
{
    assumeTrue(set.get(i));

    BitSet originalSet = (BitSet) set.clone();

    set.set(i);

    assertThat(originalSet.equals(set));
}
```

Property Based Testing

Ejemplo

“ Para todo set, si se agrega un elemento que ya pertenece al set entonces el set no cambia”

Precondición: Descripción de las entradas para las cuales la propiedad es válida

```
void addAlreadyBelongsToDoesNotChange(BitSet set, int i)
{
    assumeTrue(set.get(i));

    BitSet originalSet = (BitSet) set.clone();

    set.set(i);

    assertThat(originalSet.equals(set));
}
```

Property Based Testing

Ejemplo

“ Para todo set, si se agrega un elemento que ya pertenece al set entonces el set no cambia”

Precondición: Descripción de las entradas para las cuales la propiedad es válida

```
void addAlreadyBelongsToDoesNotChange(BitSet set, int i)
{
    assumeTrue(set.get(i));

    BitSet originalSet = (BitSet) set.clone();

    set.set(i);

    assertThat(originalSet.equals(set));
}
```

Postcondición:
Propiedad que se espera que el código satisfaga, para valores cualesquiera de los parámetros que cumplan con la precondición

Testing basado en propiedades: Parámetros y valores

Para correr un test basado en propiedades debemos proveer valores para los parámetros.

```
@ParameterizedTest
@MethodSource("queueProvider")
public void enqueueSizeProperty(BoundedQueue queue) {
    assumeTrue("cola no llena", !queue.isFull());

    int oldSize = queue.size();
    queue.enqueue(5);

    assertThat(queue.size(), is(oldSize + 1));
}
```

Testing basado en propiedades: Parámetros y valores

Para correr un test basado en propiedades debemos proveer valores para los parámetros.

Tests
Parametrizados

```
@ParameterizedTest
@MethodSource("queueProvider")
public void enqueueSizeProperty(BoundedQueue queue) {
    assertTrue("cola no llena", !queue.isFull());

    int oldSize = queue.size();
    queue.enqueue(5);

    assertThat(queue.size(), is(oldSize + 1));
}
```

Testing basado en propiedades: Parámetros y valores

Para correr un test basado en propiedades debemos proveer valores para los parámetros.

Tests
Parametrizados

```
private static BoundedQueue generateQueue(int maxSize) {  
    BoundedQueue q = new BoundedQueue();  
    java.util.Random r = new java.util.Random();  
    int numberOfElements = r.nextInt(maxSize);  
    for (int i=0; i<numberOfElements; i++) {  
        q.enqueue(r.nextInt(50));  
    }  
    return q;  
}
```

Testing basado en propiedades: Parámetros y valores

Para correr un test basado en propiedades debemos proveer valores para los parámetros.

Tests
Parametrizados

```
private static BoundedQueue generateQueue(int maxSize) {
```

```
private static Stream<BoundedQueue> queueProvider() {  
    Stream<BoundedQueue> stream = Stream.generate(  
        () -> generateQueue(5)  
    ).limit(10);  
    return stream;  
}
```

Testing basado en propiedades: Parámetros y valores

Para correr un test basado en propiedades debemos proveer valores para los parámetros.

```
@ParameterizedTest
@MethodSource("bitSetAndIntProvider")
void addAlreadyBelongsToDoesNotExtendsSet(BitSet set, int i) {

    assumeTrue(set.get(i));

    BitSet originalSet = (BitSet) set.clone();

    set.set(i);

    assertThat(originalSet.equals(set));

}
```

Testing basado en propiedades: Parámetros y valores

Para correr un test basado en propiedades debemos proveer valores para los parámetros.

```
private static BitSet generateBitSet(int maxElement) {  
    BitSet s = new BitSet();  
    java.util.Random r = new java.util.Random();  
    int numberOfElements = r.nextInt(maxElement) + 1;  
    for (int i=0; i<numberOfElements; i++) {  
        s.set(r.nextInt(20));  
    }  
    return s;  
}
```

```
assertThat(originalSet.equals(set));
```

```
}
```

Testing basado en propiedades: Parámetros y valores

Para correr un test basado en propiedades debemos proveer valores para los parámetros.

```
private static BitSet generateBitSet(int maxElement) {  
    BitSet bitSet = new BitSet(maxElement);  
    Random random = new Random();  
    for (int i = 0; i < maxElement; i++) {  
        bitSet.set(i, random.nextBoolean());  
    }  
    return bitSet;  
}  
  
private static Stream<Arguments> bitSetAndIntProvider() {  
    Stream<Arguments> stream = Stream.generate(  
        () -> Arguments.of(  
            generateBitSet(30),  
            new java.util.Random().nextInt(50)  
        )  
    ).limit(10);  
    return stream;  
}
```

Testing basado en propiedades

jqwik es una libreria para implementar test basados en propiedad en Java

```
@Property(tries=20)
void lengthOfConcatenatedStringIsGreaterThanLengthOfEach(
    @ForAll String string1, @ForAll String string2
) {

    Assume.that(string1.length() != 0);
    Assume.that(string2.length() != 0);

    String conc = string1 + string2;

    assertThat(conc.length(), greaterThan(string1.length()));
    assertThat(conc.length(), greaterThan(string2.length()));
}
```

Testing basado en propiedades con jqwik

jqwik intenta generar valores para los parámetros para los tipos:

- Object,
- String
- tipos básicos
- Boolean, Character, Integer,...
- Enum
- List<T>, Set<T>, Stream<T>, Map<K, T>, HashMap<K,T> siendo K y T algunos de los tipos en esta lista
- etc

Testing basado en propiedades

jqwik es una libreria para implementar test basados en propiedad en Java

```
@Property
void boundedQueueProperty(@ForAll List<@IntRange(min = 0, max = 10) Integer> aList,
                           @ForAll int a) {

    BoundedQueue queue = generateQueueFromList(aList);
    Assume.that(!queue.isFull());

    int oldSize = queue.size();
    queue.enqueue(a);
    assertThat(queue.size(),is(oldSize+1));

}
```

Testing basado en propiedades

jqwik es
propiedades

```
private static BoundedQueue generateQueueFromList(List<Integer> l) {  
    BoundedQueue q = new BoundedQueue();  
    int numberOfElements = l.size();  
    for (int i=0; i<numberOfElements; i++) {  
        q.enqueue(l.get(i));  
    }  
    return q;  
}
```

```
@Property  
void boundedQueueProperty(@ForAll List<@IntRange(min = 0, max = 10) Integer> aList,  
    @ForAll int a) {  
  
    BoundedQueue queue = generateQueueFromList(aList);  
    Assume.that(!queue.isFull());  
  
    int oldSize = queue.size();  
    queue.enqueue(a);  
    assertThat(queue.size(), is(oldSize+1));  
  
}
```

Testing basado en propiedades

Métodos proveedores de parámetros: Generadores

```
@Property
void boundedQueuePropertyWithGenerator(@ForAll("queueGenerator") BoundedQueue queue,
                                         @ForAll int a) {
    Assume.that(!queue.isFull());

    int oldSize = queue.size();
    queue.enqueue(a);
    assertThat(queue.size(), is(oldSize+1));
}
```

Testing basado en propiedades

Métodos

```
@Provide
private Arbitrary<BoundedQueue> queueGenerator() {
    return Arbitraries.create(()->generateQueue(30));
}
```

```
@Property
void boundedQueuePropertyWithGenerator(@ForAll("queueGenerator") BoundedQueue queue,
                                         @ForAll int a) {
    Assume.that(!queue.isFull());

    int oldSize = queue.size();
    queue.enqueue(a);
    assertThat(queue.size(), is(oldSize+1));
}
```

Testing basado en propiedades

Métodos

```
@Provide
private Arbitrary<BoundedQueue> queueGenerator() {
    return Arbitraries.create(()->generateQueue(30));
}
```

```
@Property
void boundedQueuePropertyWithGenerator(@ForAll("queueGenerator") BoundedQueue queue,
                                         @ForAll int a) {
```

```
    Assume.that(!queue.isFull());
```

```
    int oldSize = queue.size();
    queue.enqueue(a);
    assertThat(queue.size(), is(oldSize + 1));
}

private static BoundedQueue generateQueue(int maxSize) {
    BoundedQueue q = new BoundedQueue();
    java.util.Random r = new java.util.Random();
    int numberOfElements = r.nextInt(maxSize);
    for (int i=0; i<numberOfElements; i++) {
        q.enqueue(r.nextInt(50));
    }
    return q;
}
```

Documentación

jqwik:

- <https://jqwik.net/docs/current/user-guide.html>